Reglas semánticas del analizador

Proyecto #2 - Compiladores e Intérpretes - TEC - Semestre II, 2026

Cómo leer cada ficha

- **Código:** las letras son el tipo de revisión (ARI = aritmética, ARR = arreglos…) y el número es la regla.
- **Mensaje:** es el texto exacto que escribe el programa cuando la regla falla. En pantalla aparece junto a la línea y la columna del error.
- **MAL:** código que provoca el error. La línea marcada en rojo es la que reporta el programa.
- **BIEN:** el mismo código corregido, sin errores.
- Los ejemplos van dentro de la función principal. Cuando hace falta otra función, se muestra completa antes del ejemplo.

Reglas generales que valen para todas

- **1. Tipado explícito y fuerte:** toda variable se declara con su tipo y los tipos no se mezclan (un int y un float no se combinan).
- **2. Un error no se repite:** si un valor ya tiene un error, las operaciones que lo usan no producen más avisos.
- **3. El análisis continúa:** después de un error el programa sigue revisando y reporta todos los demás.
- **4. Declarar antes de usar:** las variables y las funciones deben escribirse antes de usarlas. El mismo nombre sí puede repetirse en bloques distintos.
- **5. Con errores no se genera código intermedio.**

Tres clases de fichas

- **Error (43):** la regla produce su propio mensaje de error.
- **Apoyo (8):** no produce un error propio. Explica algo que el programa hace para no repetir avisos o para poder revisar otras reglas; el ejemplo muestra su efecto.
- **Sintaxis (1):** Se detecta al leer el programa, antes de la revisión de sentido, y el análisis continúa.

Reglas semánticas del analizador

Proyecto #2 - Compiladores e Intérpretes - TEC - Semestre II, 2026

Resumen de todas las reglas

Tipo de revisión	Qué revisa	Clase
Nombres	Nombre repetido en el mismo bloque	Error
Nombres	Variable no declarada	Error
Nombres	Función no declarada o que no es una función	Error
Nombres	Arreglo no declarado	Error
Nombres	Se usa como arreglo algo que no lo es	Error
Nombres	Una función se usa como si fuera una variable	Error
Nombres	Un arreglo se usa sin su posición	Error
Aritmética	Un operando con error no repite el aviso	Apoyo
Aritmética	Operandos que no son números	Error
Aritmética	Mezclar int y float	Error
Aritmética	División entera con decimales	Error
Aritmética	División decimal con enteros	Error
Aritmética	Las constantes enteras se calculan	Apoyo
Aritmética	Potencia con exponente negativo	Error
Comparación	Igualdad entre tipos distintos	Error
Comparación	Orden entre tipos distintos	Error
Comparación	Comparar textos	Error
Comparación	Una comparación siempre da bool	Apoyo
Lógica	Operador lógico sin valores bool	Error
Lógica	Negación sin valor bool	Error
Lógica	Una operación lógica siempre da bool	Apoyo

Reglas semánticas del analizador

Proyecto #2 - Compiladores e Intérpretes - TEC - Semestre II, 2026

Tipo de revisión	Qué revisa	Clase
Asignación	Valor inicial de otro tipo	Error
Asignación	Asignar un valor de otro tipo	Error
Asignación	Un valor con error no repite el aviso	Apoyo
Arreglos	Tamaño que no es entero	Error
Arreglos	Tamaño cero	Error
Arreglos	Valores iniciales de otro tipo	Error
Arreglos	Más valores que el tamaño	Error
Arreglos	Posición que no es entera	Error
Arreglos	Posición fuera del arreglo	Error
Arreglos	Posición negativa	Error
Arreglos	Guardar en una posición un valor de otro tipo	Error
Arreglos	El tamaño puede ser una operación con constantes	Apoyo
Funciones	Cantidad de datos incorrecta en una llamada	Error
Funciones	Tipo de dato incorrecto en una llamada	Error
Funciones	Función que retorna string	Error
Funciones	Parámetro repetido	Error
Funciones	void con un nombre que no es principal	Sintaxis
Funciones	La llamada tiene el tipo de retorno de la función	Apoyo
Funciones	Función con tipo sin return	Error
Return	La función principal retorna un valor	Error
Return	El return no es del tipo de la función	Error
Return	Un return sin valor se permite	Apoyo

Reglas semánticas del analizador

Proyecto #2 - Compiladores e Intérpretes - TEC - Semestre II, 2026

Tipo de revisión	Qué revisa	Clase
Control	Condición del if que no es bool	Error
Control	Condición del while que no es bool	Error
Control	Condición del for que no es bool	Error
Control	Tipo incorrecto en la cabecera del for	Error
Control	break fuera de un ciclo	Error
Incremento	++ o -- con un tipo que no es número	Error
Entrada/Salida	read con un tipo no permitido	Error
Entrada/Salida	write con un tipo no permitido	Error
Programa	Falta la función principal	Error

Reglas semánticas del analizador

Proyecto #2 - Compiladores e Intérpretes - TEC - Semestre II, 2026

Nombres

Revisan que cada nombre se declare antes de usarse y que se use de la forma correcta: variable, función o arreglo.

Nombre repetido en el mismo bloque	
Dentro de un mismo bloque no se puede declarar dos veces el mismo nombre. Lo mismo vale para la función principal, que solo puede existir una vez.	
Mensaje que muestra el programa 'a' ya fue declarado en este ámbito (línea 2)	
MAL val int a ⊢ 1» val int a ⊢ 2»	**BIEN** val int a ⊢ 1» if ϵ: true :ɔ ¿: val int a ⊢ 2» | otro bloque: se permite :?
Cómo solucionarlo: Cambiar el nombre de una de las dos o dejar una sola declaración. El mismo nombre sí se puede usar en otro bloque distinto (por ejemplo, dentro de un if).	

Variable no declarada	
Toda variable debe declararse con val antes de usarla.	
Mensaje que muestra el programa 'a' no ha sido declarado	
MAL a ⊢ 5»	**BIEN** val int a ⊢ 0» a ⊢ 5»
Cómo solucionarlo: Declarar la variable antes de usarla, con su tipo.	

Función no declarada o que no es una función	
Solo se puede llamar a una función que exista y que se haya escrito antes. También da error si se llama como función a algo que es una variable.	
Mensaje que muestra el programa la función 'suma' no ha sido declarada	
MAL val int r ⊢ suma ϵ: 1, 2 :ə»	**BIEN** int suma ϵ: int x, int y :ə ¿: return x + y» :? val int r ⊢ suma ϵ: 1, 2 :ə»
Cómo solucionarlo: Escribir la función antes de llamarla, o revisar que el nombre esté bien escrito.	

Arreglo no declarado	
Para usar la posición de un arreglo (nombreʃ:0:ʅ) el arreglo debe haberse declarado antes.	
Mensaje que muestra el programa el arreglo 'notas' no ha sido declarado	
MAL notasʃ:0:ʅ ⊢ 5»	**BIEN** val int notasʃ:3:ʅ» notasʃ:0:ʅ ⊢ 5»
Cómo solucionarlo: Declarar el arreglo con su tipo y su tamaño antes de usarlo.	

usa como arreglo algo que no lo es	
Solo se puede escribir nombreʃ:posición:ʅ con nombres que se declararon como arreglo.	
Mensaje que muestra el programa 'x' no es un arreglo	
MAL val int x ⊢ 1» xʃ:0:ʅ ⊢ 2»	**BIEN** val int xʃ:3:ʅ» xʃ:0:ʅ ⊢ 2»
Cómo solucionarlo: Declarar el nombre como arreglo, o quitar la posición si es una variable normal.	

Una función se usa como si fuera una variable	
El nombre de una función no se puede usar como un valor. Una función se usa llamándola, con sus paréntesis y sus datos.	
Mensaje que muestra el programa 'suma' es una función y se usa como variable	
MAL int suma ϵ: int x, int y :ɜ ¿: return x + y» :? val int r ⊢ suma»	**BIEN** int suma ϵ: int x, int y :ɜ ¿: return x + y» :? val int r ⊢ suma ϵ: 1, 2 :ɜ»
Cómo solucionarlo: Llamar a la función con sus paréntesis: suma ϵ: 1, 2 :ɜ.	

Un arreglo se usa sin su posición	
Un arreglo completo no se puede usar como un valor suelto. Hay que decir cuál posición se quiere.	
Mensaje que muestra el programa 'notas' es un arreglo y debe usarse con un índice	
MAL val int notasʃ:3:ɳ» val int x ⊢ notas»	**BIEN** val int notasʃ:3:ɳ» val int x ⊢ notasʃ:0:ɳ»
Cómo solucionarlo: Agregar la posición entre ʃ: y :ɳ, por ejemplo notasʃ:0:ɳ.	

Reglas semánticas del analizador

Proyecto #2 - Compiladores e Intérpretes - TEC - Semestre II, 2026

Aritmética

Revisan las operaciones con números: + - * / // mod pot.

Un operando con error no repite el aviso [regla de apoyo]	
Si uno de los números de una operación ya tiene un error (por ejemplo, una variable que no existe), la operación no agrega otro mensaje. Así un solo problema no produce una cadena de avisos.	
Lo que muestra el programa (un solo aviso) 'zz' no ha sido declarado	
MAL val int a ⊢ 1» val int d ⊢ a + zz»	**BIEN** val int a ⊢ 1» val int zz ⊢ 2» val int d ⊢ a + zz»
Cómo solucionarlo: Corregir solo el error original: declarar la variable que falta.	

Operandos que no son números	
Las operaciones aritméticas (+ - * / // mod pot) solo trabajan con números, es decir, con int o float. No se pueden usar con textos, caracteres o bool.	
Mensaje que muestra el programa el operador '+' no se puede aplicar a 'string' y 'int' (se requieren int o float)	
MAL val int r ⊢ "hola" + 1»	**BIEN** val int r ⊢ 2 + 1»
Cómo solucionarlo: Usar solo números en la operación. Si se quiere unir textos, esta operación no existe en el lenguaje.	

Mezclar int y float	
El lenguaje es de tipado fuerte: un entero (int) y un decimal (float) no se pueden mezclar en la misma operación.	
Mensaje que muestra el programa no se pueden mezclar 'int' y 'float' con el operador '+' (tipado fuerte)	
MAL val float r ⊢ 1 + 2.5»	**BIEN** val float r ⊢ 1.0 + 2.5»
Cómo solucionarlo: Usar el mismo tipo en los dos lados. Por ejemplo, escribir 1.0 en lugar de 1.	

Reglas semánticas del analizador

Proyecto #2 - Compiladores e Intérpretes - TEC - Semestre II, 2026

División entera con decimales	
La división entera (//) solo se puede hacer con números enteros.	
Mensaje que muestra el programa el operador '//' requiere operandos int y recibió 'float'	
✗ **MAL** val float r ⊢ 5.0 // 2.0»	✓ **BIEN** val int r ⊢ 5 // 2»
Cómo solucionarlo: Usar números int con //, o usar la división decimal (/) si los números son float.	

División decimal con enteros	
La división decimal (/) solo se puede hacer con números decimales (float).	
Mensaje que muestra el programa el operador '/' requiere operandos float y recibió 'int'	
✗ **MAL** val int r ⊢ 4 / 2»	✓ **BIEN** val float r ⊢ 4.0 / 2.0»
Cómo solucionarlo: Usar números float con /, o usar la división entera (//) si los números son int.	

Las constantes enteras se calculan [regla de apoyo]	
Si los dos lados de una operación son constantes enteras, el programa calcula el resultado. Eso permite revisar tamaños y posiciones de arreglos que están escritos como operaciones.	
Lo que muestra el programa (gracias al cálculo) el índice 2 está fuera del arreglo 'notas' (índices válidos: 0 a 1)	
MAL val int notasʃ:1 + 1:ʅ» val int x ⊢ notasʃ:2:ʅ»	**BIEN** val int notasʃ:1 + 1:ʅ» val int x ⊢ notasʃ:1:ʅ»
Cómo solucionarlo: No hay nada que corregir en la operación: es un apoyo para otras reglas. En el ejemplo, 1 + 1 vale 2, así que la posición 2 no existe.	

Potencia con exponente negativo	
La potencia (pot) solo acepta un exponente positivo.	
Mensaje que muestra el programa el operador 'pot' requiere un exponente positivo	
MAL val int r ⊢ 2 pot -1»	**BIEN** val int r ⊢ 2 pot 3»
Cómo solucionarlo: Usar un exponente positivo.	

Comparación

Revisan las comparaciones: < > <= >= == !=.

Igualdad entre tipos distintos	
Con == y != se pueden comparar int, float, bool y char, pero los dos lados deben ser del mismo tipo (o los dos numéricos).	
Mensaje que muestra el programa no se puede comparar 'bool' con 'int' usando '=='	
MAL val bool b ⊢ ʃ:true:ɻ == ʃ:1:ɻ»	**BIEN** val bool b ⊢ ʃ:1:ɻ == ʃ:1:ɻ»
Cómo solucionarlo: Comparar valores del mismo tipo.	

Orden entre tipos distintos	
Con <, >, <= y >= los dos lados deben ser int o los dos float. No se mezclan.	
Mensaje que muestra el programa no se puede comparar 'int' con 'float' usando '<'	
MAL val bool b ⊢ ʃ:1:ɻ < ʃ:2.5:ɻ»	**BIEN** val bool b ⊢ ʃ:1:ɻ < ʃ:2:ɻ»
Cómo solucionarlo: Usar el mismo tipo numérico en los dos lados.	

Reglas semánticas del analizador

Proyecto #2 - Compiladores e Intérpretes - TEC - Semestre II, 2026

Comparar textos	
Los textos (string) no se pueden comparar con ninguna comparación.	
Mensaje que muestra el programa no se puede comparar 'string' con 'string' usando '=='	
MAL val bool b ⊢ ʃ:"a":ɻ == ʃ:"b":ɻ»	**BIEN** val bool b ⊢ ʃ:'a':ɻ == ʃ:'b':ɻ»
Cómo solucionarlo: Comparar caracteres (char) o números, no textos.	

Una comparación siempre da bool [regla de apoyo]	
El resultado de una comparación es siempre de tipo bool, aunque la comparación tenga un error. Así, si se guarda en una variable de otro tipo, el aviso es claro y no se encadenan errores.	
Lo que muestra el programa la variable 'x' es de tipo 'int' y se le da un valor de tipo 'bool'	
MAL val int a ⊢ 5» val int x ⊢ ʃ:a:ɻ < ʃ:10:ɻ»	**BIEN** val int a ⊢ 5» val bool x ⊢ ʃ:a:ɻ < ʃ:10:ɻ»
Cómo solucionarlo: Guardar el resultado de la comparación en una variable bool.	

Lógica

Revisan los operadores λ (y), θ (o) y Σ (no).

Operador lógico sin valores bool	
Los operadores λ (y) y θ (o) solo trabajan con valores verdaderos o falsos (bool).	
Mensaje que muestra el programa el operador lógico 'and' requiere operandos bool y recibió 'int' y 'bool'	
MAL val bool b ⊢ ϵ: 5 :ə λ ϵ: true :ə»	**BIEN** val bool b ⊢ ϵ: true :ə λ ϵ: false :ə»
Cómo solucionarlo: Usar comparaciones o valores bool a los dos lados.	

Negación sin valor bool
El operador Σ (no) solo trabaja con valores verdaderos o falsos. Con la gramática actual la negación solo recibe condiciones, así que esta regla funciona como seguridad y no se puede provocar escribiendo código.
BIEN (ejemplo de uso correcto) val int a ⊢ 1» val bool b ⊢ Σ ɛ: ʃːaːɻ < ʃːɔːɻ :ə»
Cómo solucionarlo: Usar Σ solo con condiciones, por ejemplo Σ ɛ: ʃːaːɻ < ʃːbːɻ :ə. No se puede mostrar un ejemplo malo: la gramática no permite escribir una negación con algo que no sea una condición.

Una operación lógica siempre da bool [regla de apoyo]	
El resultado de una operación lógica (λ, θ) es siempre de tipo bool.	
Lo que muestra el programa la variable 'v' es de tipo 'int' y se le da un valor de tipo 'bool'	
MAL val int a ⊢ 5» val int v ⊢ ɛːʃːaːɻ > ʃːɔːɻːə λ ɛːʃːaːɻ < ʃːɔːɻːə»	**BIEN** val int a ⊢ 5» val bool v ⊢ ɛːʃːaːɻ > ʃːɔːɻːə λ ɛːʃːaːɻ < ʃːɔːɻːə»
Cómo solucionarlo: Guardar el resultado en una variable bool.	

Asignación

Revisan que un valor se guarde en una variable de su mismo tipo.

Valor inicial de otro tipo	
El valor con que se crea una variable debe tener el mismo tipo que la variable.	
Mensaje que muestra el programa la variable 'm' es de tipo 'int' y se le da un valor de tipo 'float'	
MAL val int m ⊢ 2.5»	**BIEN** val float m ⊢ 2.5»
Cómo solucionarlo: Cambiar el tipo de la variable o cambiar el valor.	

Asignar un valor de otro tipo	
Cuando se cambia el valor de una variable ya creada, el nuevo valor debe ser del mismo tipo.	
Mensaje que muestra el programa la variable 'm' es de tipo 'int' y se le da un valor de tipo 'float'	
MAL val int m ⊢ 1» m ⊢ 2.5»	**BIEN** val int m ⊢ 1» m ⊢ 2»
Cómo solucionarlo: Asignar un valor del tipo de la variable.	

Un valor con error no repite el aviso [regla de apoyo]	
Si el valor que se quiere guardar ya tiene un error, o la variable ya falló, la asignación no agrega otro mensaje.	
Lo que muestra el programa (un solo aviso) el arreglo 'noExiste' no ha sido declarado	
mAL val int y ⊢ noExisteʃ:0:ʅ»	**BIEN** val int noExisteʃ:1:ʅ» val int y ⊢ noExisteʃ:0:ʅ»
Cómo solucionarlo: Corregir solo el error original: declarar el arreglo que falta.	

Arreglos

Revisan cómo se crean y se usan los arreglos: su tamaño, sus valores y sus posiciones.

Tamaño que no es entero	
El tamaño de un arreglo debe ser un número entero (int).	
Mensaje que muestra el programa el tamaño del arreglo 'notas' debe ser int y se encontró 'float'	
MAL val int notasʃ:2.5:ʅ»	**BIEN** val int notasʃ:3:ʅ»
Cómo solucionarlo: Escribir el tamaño como un entero.	

Tamaño cero	
El tamaño de un arreglo debe ser mayor que cero.	
Mensaje que muestra el programa el tamaño del arreglo 'notas' debe ser mayor que 0 y es 0	
MAL val int notasʃ:0:ʅ»	**BIEN** val int notasʃ:1:ʅ»
Cómo solucionarlo: Escribir un tamaño de 1 o más.	

Valores iniciales de otro tipo	
Todos los valores con que se crea un arreglo deben ser del tipo del arreglo.	
Mensaje que muestra el programa el arreglo 'notas' es de tipo 'int' pero 1 de sus 3 valores no lo son (el primero es el elemento 1, de tipo 'float')	
MAL val int notasʃ:3:ʅ ⊢ ¿:1, 2.5, 3:?»	**BIEN** val int notasʃ:3:ʅ ⊢ ¿:1, 2, 3:?»
Cómo solucionarlo: Usar solo valores del tipo del arreglo.	

Más valores que el tamaño	
No se pueden dar más valores iniciales que el tamaño del arreglo.	
Mensaje que muestra el programa el arreglo 'notas' tiene tamaño 2 pero se dieron 3 valores	
MAL val int notasʃ:2:ʅ ⊢ ¿:1, 2, 3:?»	**BIEN** val int notasʃ:3:ʅ ⊢ ¿:1, 2, 3:?»
Cómo solucionarlo: Aumentar el tamaño o quitar valores.	

Posición que no es entera	
La posición de un arreglo debe ser un número entero.	
Mensaje que muestra el programa el índice del arreglo 'notas' debe ser int y se encontró 'float'	
MAL val int notasʃ:3:ɻ» val int x ⊢ notasʃ:1.5:ɻ»	**BIEN** val int notasʃ:3:ɻ» val int x ⊢ notasʃ:1:ɻ»
Cómo solucionarlo: Usar un entero como posición.	

Posición fuera del arreglo	
Un arreglo de tamaño n tiene las posiciones de 0 a n-1. Usar una posición mayor da error.	
Mensaje que muestra el programa el índice 5 está fuera del arreglo 'notas' (índices válidos: 0 a 2)	
MAL val int notasʃ:3:ɻ» val int x ⊢ notasʃ:5:ɻ»	**BIEN** val int notasʃ:3:ɻ» val int x ⊢ notasʃ:2:ɻ»
Cómo solucionarlo: Usar una posición que exista, o hacer el arreglo más grande.	

Posición negativa	
Las posiciones empiezan en 0, por lo que no pueden ser negativas.	
Mensaje que muestra el programa el índice -1 del arreglo 'notas' no puede ser negativo	
MAL val int notasʃ:3:ɻ» val int x ⊢ notasʃ:-1:ɻ»	**BIEN** val int notasʃ:3:ɻ» val int x ⊢ notasʃ:0:ɻ»
Cómo solucionarlo: Usar una posición de 0 en adelante.	

Guardar en una posición un valor de otro tipo	
Al asignar a una posición del arreglo, el valor debe ser del tipo de los elementos.	
Mensaje que muestra el programa la variable 'notas[0]' es de tipo 'int' y se le da un valor de tipo 'float'	
MAL val int notasʃ:3:ʅ» notasʃ:0:ʅ ⊢ 2.5»	**BIEN** val int notasʃ:3:ʅ» notasʃ:0:ʅ ⊢ 25»
Cómo solucionarlo: Asignar un valor del tipo del arreglo. En la versión del programa que genera código intermedio para arreglos, el nombre incluye la posición (notas[0]). En una versión anterior aparece solo notas.	

El tamaño puede ser una operación con constantes [regla de apoyo]
El tamaño de un arreglo puede escribirse como una operación con constantes. El programa la calcula y la tabla de símbolos muestra el resultado.
Lo que muestra el programa No hay error. La tabla de símbolos muestra la dimensión [0..1].
BIEN (ejemplo de uso correcto) val int szʃ:1 + 1:ʅ»
Cómo solucionarlo: No hay nada que corregir.

Funciones

Revisan cómo se escriben las funciones y cómo se llaman.

Cantidad de datos incorrecta en una llamada	
Al llamar una función hay que dar exactamente los datos (argumentos) que pide.	
Mensaje que muestra el programa la función 'suma' espera 2 argumento(s) y recibió 1	
MAL int suma ϵ: int x, int y :ɜ ¿: return x + y» :? val int r ⊢ suma ϵ: 1 :ɜ»	**BIEN** int suma ϵ: int x, int y :ɜ ¿: return x + y» :? val int r ⊢ suma ϵ: 1, 2 :ɜ»
Cómo solucionarlo: Dar la cantidad de datos que la función necesita.	

Tipo de dato incorrecto en una llamada	
Cada dato que se le da a la función debe ser del tipo del parámetro correspondiente.	
Mensaje que muestra el programa el argumento 2 de 'suma' debe ser 'int' y se encontró 'float'	
MAL int suma ϵ: int x, int y :ɜ ¿: return x + y» :? val int r ⊢ suma ϵ: 1, 2.5 :ɜ»	**BIEN** int suma ϵ: int x, int y :ɜ ¿: return x + y» :? val int r ⊢ suma ϵ: 1, 2 :ɜ»
Cómo solucionarlo: Dar un dato del tipo que la función espera.	

Función que retorna string	
Las funciones solo pueden retornar int, float, char o bool. Un string no se puede retornar.	
Mensaje que muestra el programa la función 'saludo' no puede retornar string (solo int, float, char o bool)	
MAL string saludo ϵ: :ə ¿: return "hola"» :? val int a ⊢ 1»	**BIEN** char letra ϵ: :ə ¿: return 'a'» :? val int a ⊢ 1»
Cómo solucionarlo: Cambiar el tipo de retorno a uno permitido.	

Parámetro repetido	
Los parámetros de una función no pueden tener el mismo nombre.	
Mensaje que muestra el programa 'x' ya fue declarado en este ámbito (línea 1)	
MAL int f ϵ: int x, int x :ə ¿: return x» :? val int a ⊢ 1»	**BIEN** int f ϵ: int x, int y :ə ¿: return x + y» :? val int a ⊢ 1»
Cómo solucionarlo: Cambiar el nombre de uno de los parámetros.	

void con un nombre que no es principal [error de sintaxis]	
La palabra void solo se puede usar en la función principal. Una función void con otro nombre es un error de sintaxis: el programa lo reporta y sigue analizando (modo pánico).	
Lo que muestra el programa (error de sintaxis) Error sintáctico en la línea 1, columna 6. Se encontró: 'otra' (ID). Se esperaba: principal	
MAL void otra ϵ: :ə ¿: :? val int a ⊢ 1»	**BIEN** int otra ϵ: :ə ¿: return 1» :? val int a ⊢ 1»
Cómo solucionarlo: Cambiar void por un tipo de retorno (int, float, char o bool) y agregar su return.	

La llamada tiene el tipo de retorno de la función [regla de apoyo]	
Una llamada a una función vale lo mismo que el tipo que devuelve la función. Si se guarda en una variable de otro tipo, el aviso es de asignación.	
Lo que muestra el programa la variable 'e' es de tipo 'int' y se le da un valor de tipo 'float'	
MAL float mitad ϵ: float x :ə ¿: return x / 2.0» :? val int e ⊢ mitad ϵ: 3.0 :ə»	**BIEN** float mitad ϵ: float x :ə ¿: return x / 2.0» :? val float e ⊢ mitad ϵ: 3.0 :ə»
Cómo solucionarlo: Guardar el resultado en una variable del tipo que devuelve la función.	

Función con tipo sin return	
Toda función que tiene un tipo de retorno debe tener al menos un return.	
Mensaje que muestra el programa la función 'f' debe tener al menos un return	
MAL int f ϵ: :ə ¿: val int a ⊢ 1» :? val int a ⊢ 1»	**BIEN** int f ϵ: :ə ¿: val int a ⊢ 1» return a» :? val int a ⊢ 1»
Cómo solucionarlo: Agregar un return con un valor del tipo de la función.	

Return

Revisan lo que retorna cada función.

La función principal retorna un valor	
La función principal es void: no devuelve nada, así que no puede tener return con un valor.	
Mensaje que muestra el programa el procedimiento 'principal' no puede retornar un valor	
MAL return 5»	**BIEN** return»
Cómo solucionarlo: Quitar el valor del return o quitar el return.	

El return no es del tipo de la función	
El valor de un return debe tener el mismo tipo que declaró la función.	
Mensaje que muestra el programa la función 'f' debe retornar un valor de tipo int, pero se recibió float	
MAL int f ϵ: :ə ¿: return 2.5» :? val int a ⊢ 1»	**BIEN** int f ϵ: :ə ¿: return 2» :? val int a ⊢ 1»
Cómo solucionarlo: Retornar un valor del tipo de la función, o cambiar el tipo de la función.	

Un return sin valor se permite [regla de apoyo]
Un return sin valor es válido: sirve para terminar una función antes de llegar a su final.
Lo que muestra el programa No hay error.
BIEN (ejemplo de uso correcto) return»
Cómo solucionarlo: No hay nada que corregir.

Control

Revisan las condiciones de if, elif, while y for, y el uso de break.

Condición del if que no es bool	
La condición de un if o de un elif debe ser verdadera o falsa (bool). Un número no sirve como condición.	
Mensaje que muestra el programa la condición debe ser de tipo bool, pero se recibió int	
MAL val int x ⊢ 1» if ϵ: x :ə ¿: x ⊢ 2» :?	**BIEN** val int x ⊢ 1» if ϵ: ʃ:xːɻ > ʃ:0ːɻ :ə ¿: x ⊢ 2» :?
Cómo solucionarlo: Usar una comparación (por ejemplo ʃ:xːɻ > ʃ:0ːɻ) o una variable bool.	

Condición del while que no es bool	
La condición de un while debe ser verdadera o falsa (bool).	
Mensaje que muestra el programa la condición debe ser de tipo bool, pero se recibió int	
MAL val int x ⊢ 1» while ϵ: x :ə ¿: x ⊢ 0» :?	**BIEN** val int x ⊢ 1» while ϵ: ʃ:xːɻ > ʃ:0ːɻ :ə ¿: x ⊢ 0» :?
Cómo solucionarlo: Usar una comparación o una variable bool.	

Condición del for que no es bool	
La condición que está en el centro de la cabecera de un for debe ser verdadera o falsa (bool).	
Mensaje que muestra el programa la condición debe ser de tipo bool, pero se recibió int	
MAL for ϵ: val int i ⊢ 0 » i » i++ :ə ¿: :?	**BIEN** for ϵ: val int i ⊢ 0 » ʃ:iːɳ < ʃ:3ːɳ » i++ :ə ¿: :?
Cómo solucionarlo: Usar una comparación en el centro de la cabecera.	

Tipo incorrecto en la cabecera del for	
El valor inicial y el incremento de un for deben tener el tipo de la variable de control.	
Mensaje que muestra el programa la variable 'j' es de tipo 'int' y se le da un valor de tipo 'float'	
MAL for ϵ: val int j ⊢ 0.5 » ʃ:jːɳ < ʃ:3ːɳ » j++ :ə ¿: :?	**BIEN** for ϵ: val int j ⊢ 0 » ʃ:jːɳ < ʃ:3ːɳ » j++ :ə ¿: :?
Cómo solucionarlo: Usar un valor del tipo de la variable.	

break fuera de un ciclo	
La instrucción break solo se puede usar dentro de un while o de un for.	
Mensaje que muestra el programa break solo puede utilizarse dentro de un ciclo	
MAL break»	**BIEN** while ϵ: true :ə ¿: break» :?
Cómo solucionarlo: Mover el break dentro de un ciclo, o quitarlo.	

Reglas semánticas del analizador

Proyecto #2 - Compiladores e Intérpretes - TEC - Semestre II, 2026

Incremento

Revisan los operadores ++ y --.

++ o -- con un tipo que no es número	
Los operadores ++ y -- solo se pueden usar con variables int o float.	
Mensaje que muestra el programa el operador '++' solo se aplica a variables int o float y 's' es de tipo string	
MAL val string s ⊢ "a"» s++	**BIEN** val int n ⊢ 1» n++
Cómo solucionarlo: Usar ++ y -- solo con números.	

Entrada/Salida

Revisan las instrucciones read (leer) y write (escribir).

read con un tipo no permitido	
La instrucción read solo guarda datos en variables int, float o char.	
Mensaje que muestra el programa read solo permite valores de tipo int, float o char; se recibió string	
MAL val string s ⊢ "a"» read ϵ: s :ɔ»	**BIEN** val int n ⊢ 0» read ϵ: n :ɔ»
Cómo solucionarlo: Leer en una variable de un tipo permitido.	

write con un tipo no permitido	
La instrucción write solo escribe valores int, float o string. No escribe bool ni char.	
Mensaje que muestra el programa write no permite valores de tipo bool	
MAL write ϵ: true :ɔ»	**BIEN** write ϵ: "hola" :ɔ»
Cómo solucionarlo: Escribir un valor de un tipo permitido.	

Reglas semánticas del analizador

Proyecto #2 - Compiladores e Intérpretes - TEC - Semestre II, 2026

Programa

Revisa la estructura general del programa.

Falta la función principal	
Todo programa debe tener exactamente una función llamada principal. Si se escriben dos, la segunda aparece como un nombre repetido (TS-1).	
Mensaje que muestra el programa el programa debe tener una función principal	
MAL int suma ϵ: int x :ɜ ¿: return x» :?	**BIEN** int suma ϵ: int x :ɜ ¿: return x» :? void principal ϵ: :ɜ ¿: val int r ⊢ suma ϵ: 5 :ɜ» :?
Cómo solucionarlo: Agregar la función principal.	